Bézier Curve — Matrix Method (Straight)

$$P(u) = U \cdot M \cdot G$$

Code Explanation for Julia Beginners

`04-bezier-matrix-straight.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the Julia code in `04-bezier-matrix-straight.jl` for readers new to Julia. The majority of this script is identical to script 03. This document focuses on what changed: the `section()` helper that replaces `print_step_header()`, the five new focused print functions, and one small change to `print_labeled_matrix`. Shared components are listed with a reference to `03-bezier-matrix-pedagogical-code-explained` rather than repeated.

Contents

1 Type Map: How the Pieces Fit Together **2**

2 What Is Identical to Script 03 **3**

3 What Changed: `print_labeled_matrix` **3**

4 What Is New: `section()` **4**

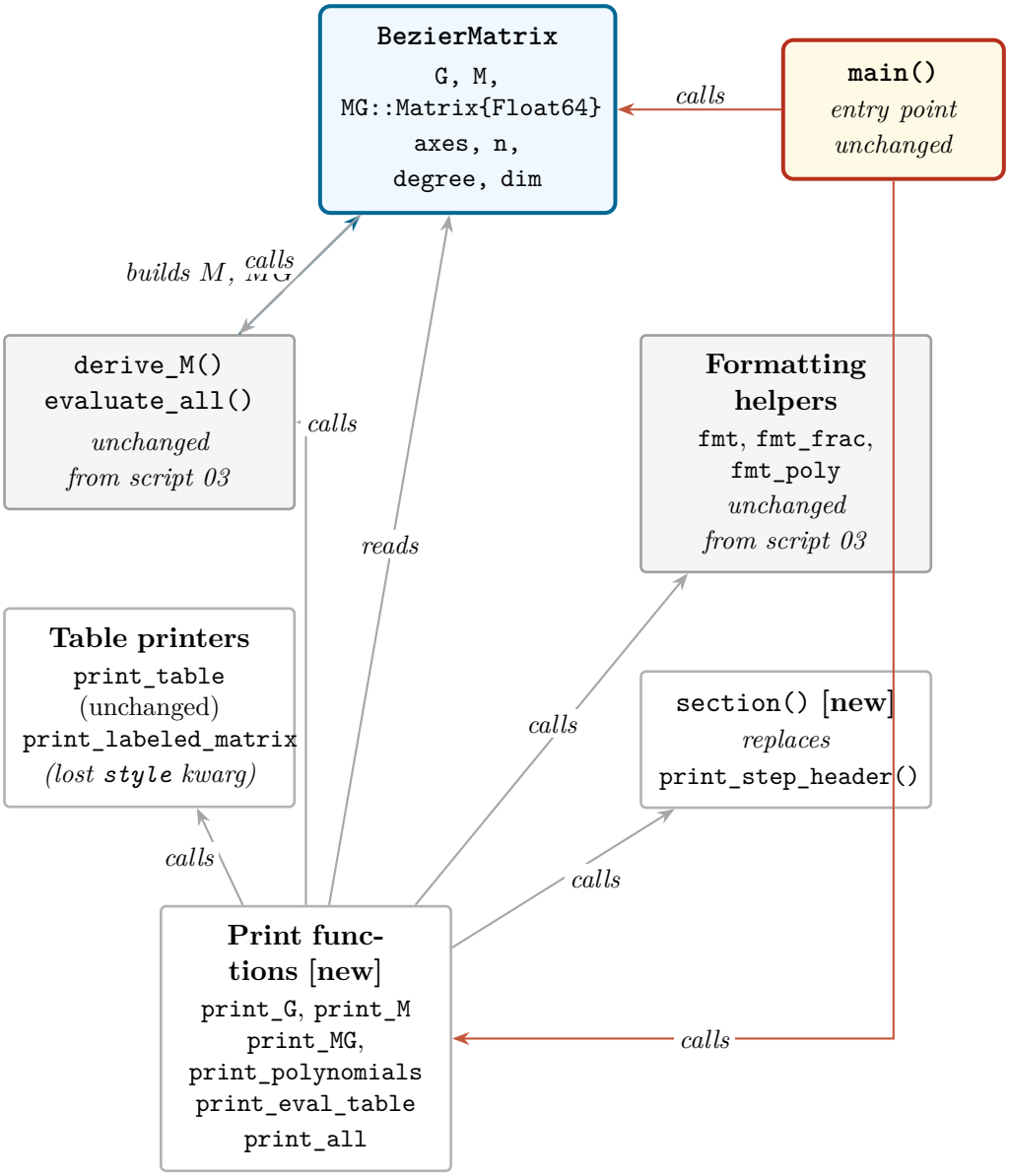
5 What Is New: The Five Print Functions **4**

5.1 `print_G` 4
5.2 `print_M` 5
5.3 `print_MG` 5
5.4 `print_polynomials` 6
5.5 `print_eval_table` 6
5.6 `print_all` 7

6 Design Principle: One Function, One Thing 7

1. Type Map: How the Pieces Fit Together

The computation core is unchanged from script 03. Only the print layer changed — the five numbered step functions were replaced by five focused, single-purpose matrix printers.



Legend: Grey-shaded boxes are components unchanged from script 03. White boxes are new or modified. The computation core (`BezierMatrix`, `derive_M`, `evaluate_all`, formatting helpers, `print_table`) is entirely reused. Only the print layer changed.

2. What Is Identical to Script 03

The following components are copied verbatim from script 03. They are documented fully in `03-bezier-matrix-pedagogical-code-explained`.

Component	Notes
`CASE_DATA`	Same control points, same u values.
`BezierMatrix` struct	Fields `G`, `M`, `MG`, `axes`, `n`, `degree`, `dim` — unchanged.
`BezierMatrix` constructor	Calls `derive_M`, computes `M * points`.
`derive_M`	Double loop, index arithmetic `M[n-k+1, i+1]`, `zeros()` initialisation.
`evaluate_all`	2-D array comprehension, one batch matrix multiply `Uall * bm.MG`.
`fmt`	Integer or 4 d.p.
`fmt_frac`	Float → rational string, short-circuit `||` guard.
`fmt_poly`	Coefficient vector → polynomial string, reversed power order.
`print_table`	Keyword args, `Union{...,Nothing}`, `:simple`/`:outline` styles.
`ruler`	66-character divider.
`SUPERSCRIPTS`	Superscript character dict.
`main`	`CASE_DATA[CASE]` → `BezierMatrix` → `print_all`.

3. What Changed: `print_labeled_matrix`

Script 03:

```
function print_labeled_matrix(
        mat        ::Matrix{Float64},
        row_labels ::Vector{String},
        col_labels ::Vector{String};
        fmt_fn     = fmt,
        style      ::Symbol = :simple)
    rows = [[fmt_fn(mat[r,c]) for c in 1:size(mat,2)]
            for r in 1:size(mat,1)]
    print_table(rows, col_labels; row_labels=row_labels, style=style)
end
```

Script 04:

```
function print_labeled_matrix(
        mat        ::Matrix{Float64},
        row_labels ::Vector{String},
        col_labels ::Vector{String};
        fmt_fn     = fmt)
    rows = [[fmt_fn(mat[r,c]) for c in 1:size(mat,2)]
```

```
            for r in 1:size(mat,1)]
    print_table(rows, col_labels; row_labels=row_labels)
end
```

The `style` keyword argument is gone. Script 03 needed it because `print_labeled_matrix` was called from both simple-table contexts (matrix display) and potentially outline contexts. Script 04 only ever calls `print_labeled_matrix` for the three matrices (G, M, $M \cdot G$), which all use the default `:simple` style. The outline table is only used in `print_eval_table`, which calls `print_table` directly with `style=:outline`.

Removing an unused parameter is good practice — it simplifies the interface and removes a source of confusion.

4. What Is New: `section()`

```
section(title::String) = (
    println();
    println("  $title");
    println("  " * "-"^(WIDTH-2))
)
```

This replaces `print_step_header(step::Int, title::String)` from script 03.

The difference is small but meaningful:

	Script 03	Script 04
Signature	`(step::Int, title::String)`	`(title::String)`
Output	`STEP 2 -- Derive Basis Matrix...`	`M -- Basis Matrix (derived from Bernste`

Script 03 numbered its steps because the output was a structured walkthrough. Script 04 has no steps — each section is an independent block labelled by its content, not by a sequence number.

Script 03:

```
STEP 2 - Derive Basis Matrix M from Bernstein Polynomials
----------------------------------------------------------------
```

Script 04:

```
M  - Basis Matrix (derived from Bernstein)
----------------------------------------------------------------
```

Same dashed rule, same indentation. Just no step number, and the title directly names the object being displayed.

5. What Is New: The Five Print Functions

Script 03 had five numbered ***print_stepN*** *functions, each performing a major computation or derivation and printing detailed working. Script 04 replaces them with five focused functions — each one prints exactly one thing.*

5.1. print_G

```
function print_G(bm::BezierMatrix)
    section("G  - Geometry Matrix (Control Points)")
    println()
    print_labeled_matrix(bm.G, ["P$(i-1)" for i in 1:bm.n], bm.axes)
end
```

Input: reads `bm.G`, `bm.n`, `bm.axes`.

What it does: prints a section header, then calls `print_labeled_matrix` with row labels `["P0","P1","P2","P3","P4"]` and column labels from `bm.axes`.

Output:

```
G  - Geometry Matrix (Control Points)
-----------------------------------------------------------------


    x  y  z
    -  -  -
P0  0  0  1
P1  0  4  1
P2  4  0  1
P3  4  4  1
P4  5  4  1
```

5.2. print_M

```
function print_M(bm::BezierMatrix)
    section("M  - Basis Matrix (derived from Bernstein)")
    n = bm.degree
    println()
    print_labeled_matrix(bm.M,
        ["u^$(n-k)" for k in 0:n],
        ["P$(i-1)" for i in 1:bm.n])
end
```

Input: reads `bm.M`, `bm.degree`, `bm.n`.

What it does: prints the basis matrix with row labels `["u^4","u^3","u^2","u^1","u^0"]` (built by the comprehension `["u^$(n-k)" for k in 0:n]`) and column labels `["P0",..."P4"]`.

The column labels here are the control point names, not the axis names — because columns of M correspond to Bernstein basis polynomials indexed by control point, not to spatial axes.

Output:

```
M  - Basis Matrix (derived from Bernstein)
----------------------------------------------------------------

      P0   P1   P2  P3  P4
      --  ---  ---  --  --
u^4    1   -4    6  -4   1
u^3   -4   12  -12   4   0
u^2    6  -12    6   0   0
u^1   -4    4    0   0   0
u^0    1    0    0   0   0
```

5.3. print_MG

```
function print_MG(bm::BezierMatrix)
    section("M * G  - Polynomial Coefficient Matrix")
    n = bm.degree
    println()
    print_labeled_matrix(bm.MG,
        ["u^$(n-k)" for k in 0:n],
        bm.axes)
end
```

Input: reads `bm.MG`, `bm.degree`, `bm.axes`.

What it does: prints the pre-computed $M \cdot G$ matrix with row labels for powers of u and column labels from `bm.axes` (the spatial axes x, y, z).

Note the column labels here switch back to `bm.axes`, because columns of $M \cdot G$ correspond to axes, not to control points.

Output:

```
M * G  - Polynomial Coefficient Matrix
----------------------------------------------------------------

        x    y  z
      ---  ---  -
u^4    13  -28  0
u^3   -32   64  0
u^2    24  -48  0
u^1     0   16  0
u^0     0    0  1
```

5.4. print_polynomials

```
function print_polynomials(bm::BezierMatrix)
    section("Resulting Polynomials")
    println()
    for (d, axis) in enumerate(bm.axes)
        println("  $(axis)(u)  =  $(fmt_poly(bm.MG[:,d], bm.degree))")
    end
end
```

Input: reads `bm.MG`, `bm.degree`, `bm.axes`.

What it does: loops over axes. For each axis `d`, takes the column slice `bm.MG[:,d]` — a `Vector{Float64}` of monomial coefficients from highest power down — and passes it to `fmt_poly` along with `bm.degree` to produce the polynomial string.

> **For axis "x" (d=1):**
>
> `bm.MG[:,1] = [13.0, -32.0, 24.0, 0.0, 0.0]`
>
> `fmt_poly([13.0, -32.0, 24.0, 0.0, 0.0], 4)` → `"13u^4 - 32u^3 + 24u^2"`
>
> Printed as: `x(u) = 13u^4 - 32u^3 + 24u^2`

Output:

```
Resulting Polynomials
----------------------------------------------------------------


x(u)  =  13u^4  - 32u^3  + 24u^2
y(u)  =  -28u^4  + 64u^3  - 48u^2  + 16u
z(u)  =  1
```

5.5. print_eval_table

```julia
function print_eval_table(bm::BezierMatrix,
                          u_values::Vector{Float64})
    section("Evaluation Table  -  P(u) = U * (M * G)")
    pts  = evaluate_all(bm, u_values)
    rows = Vector{Vector{String}}()
    for (i, u) in enumerate(u_values)
        row = [fmt(u)]
        for d in 1:bm.dim
            v = pts[i, d]
            push!(row, "$(fmt_frac(v))  ($(@sprintf("%.4f", v)))")
        end
        push!(rows, row)
    end
    println()
    print_table(rows,
        vcat(["u"], ["$(ax)(u)" for ax in bm.axes]);
        style=:outline)
end
```

Input: reads `bm.dim`, `bm.axes`, and the external `u_values` vector.

What it does: identical logic to `print_step5_table` in script 03. Calls `evaluate_all` for the batch result, builds the rows vector with `fmt_frac` + `@sprintf` cells, and prints with `style=:outline`.

The only difference from script 03 is the section header: `section("Evaluation Table ...")` instead of `print_step_header(5, "Summary ...")`.

Output:

```
Evaluation Table  -  P(u) = U * (M * G)
----------------------------------------------------------------
```

```
+==============================================================+
| u       x(u)                y(u)                z(u)         |
|==============================================================|
| 0       0  (0.0000)         0  (0.0000)   1  (1.0000)        |
| 0.2500  269//256  (1.0508)  121//64 (1.8906) 1  (1.0000)     |
| 0.5000  45//16  (2.8125)    9//4  (2.2500)   1  (1.0000)     |
| 0.7500  1053//256 (4.1133)  201//64 (3.1406) 1  (1.0000)     |
| 1       5  (5.0000)         4  (4.0000)       1  (1.0000)    |
+==============================================================+
```

5.6. print_all

```julia
function print_all(bm::BezierMatrix,
                   u_values::Vector{Float64}, case_num::Int)
    print_title(bm, case_num)
    print_G(bm)
    print_M(bm)
    print_MG(bm)
    print_polynomials(bm)
    print_eval_table(bm, u_values)
    println()
    println(ruler())
    println()
end
```

What it does: orchestrates the five print functions in order, with title and closing ruler.

Compare to script 03's `print_all`:

Script 03	Script 04
`print_title`	`print_title`
`print_control_points`	*(replaced by* ***`print_G`****)*
`print_step1_define`	*(dropped — no formulation block)*
`print_step2_derive_M`	`print_M`
`print_step3_MG`	`print_MG` + `print_polynomials`
`print_step4_evaluate`	*(dropped — no dot product display)*
`print_step5_table`	`print_eval_table`

Why split `print_step3_MG` into two functions?

In script 03, `print_step3_MG` printed both the $M \cdot G$ matrix and the resulting polynomial strings in one block under the same step header. Script 04 separates them: `print_MG` prints the matrix, `print_polynomials` prints the strings, each with its own section header. This gives the polynomials their own visual prominence — they are the practical result the user cares about most.

6. Design Principle: One Function, One Thing

The real lesson of this script is not in any single function — it is in how the functions are organised.

Script 03 organised its output around the *process*: Step 1 defines, Step 2 derives, Step 3 computes, and so on. Each step function did multiple things: explained, computed, and printed.

Script 04 organises its output around the *data*: one function per matrix, one function for the polynomials, one function for the table. Each print function does exactly one thing: print one object.

This is a principle called **separation of concerns**. Each function has a single, clear responsibility. If you want to change how the M matrix is displayed, you edit `print_M` and nothing else. If you want to add a new section, you write a new function and add one line to `print_all`.

Adding a new section would be trivial. Suppose you wanted to print the U vector for $u = 0.5$ as a new section. You would write:

```
function print_U(bm::BezierMatrix, u::Float64)
    section("U  - Parameter Vector at u = $u")
    n = bm.degree
    Uv = [u^(n-k) for k in 0:n]
    println()
    # ... format and print Uv
end
```

Then add `print_U(bm, 0.5)` to `print_all`. No other function needs to change.

Code explanation for `04-bezier-matrix-straight.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated